IND320 - Part 3: Data quality

Sipan · Report, 2 November 2026

GitHub repository · Part 3 development branch · Published app

This notebook documents regional weather sampling, a complete ERA5 download for NO3 in 2024, and four data-quality analyses. The Part 3 app pages were developed on a separate branch, tested locally and published on 28 October 2026. An explicit merge commit incorporated the changes into main, and the same public Streamlit address is retained. VASS selections use the assignment's east, west and north administrative unions. National sampling uses the union of the electricity-area polygons, with its own 1,000-point sample; a full national weather download has not yet been run. The work log records the work completed with AI assistance. The screencast is linked in Section 10.

Code cell 1

```
from IPython import get_ipython
get_ipython().run_line_magic('matplotlib', 'inline')

from pathlib import Path
import json
import sys
import hashlib
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
from IPython.display import display

# Use the same modules from the notebook and Streamlit rather than duplicate analyses.
project_root = Path.cwd() if Path('ind320').is_dir() else Path.cwd().parent
sys.path.insert(0, str(project_root))
from ind320.quality_analysis import precipitation_spc, temperature_lof, filling_stl, filling_spectrogram
print('Python:', sys.version.split()[0])
print('NumPy:', np.__version__)
```

Output

```
Python: 3.13.14
NumPy: 2.3.5
```

1. Region boundaries and spatial sampling

NVE Atlas identifies five electricity price areas, NO1-NO5. The NVE administrative layer contains RS, RØ, RV, RM and RN. Its separate download portal was unavailable when this work started. `ind320/region_data.py` therefore retrieves the original GeoJSON from the published map services and exports equivalent shapefiles locally. Source URLs, retrieval times and SHA-256 hashes are retained. These shapefiles are local exports, rather than original ZIP archives from the download portal.

The original polygons are sampled with 1,000 points each. Points are uniform by projected area in EPSG:3035, using a fixed seed and a stable seed for each region. Rejection sampling respects polygon holes and islands. Every point is checked against its source polygon before conversion to WGS84.

The assignment explicitly groups RS+RØ as east and RM+RN as north. These polygons are dissolved before drawing a fresh 1,000 points per combined region; concatenating equal-sized component samples would overweight the smaller component. RV supplies the western region. The original ten samples and these three additional samples are saved separately by region type.

Weather-region mapping for this assignment: VASS1 selects EAST (RS+RØ), VASS2 selects WEST (RV), and VASS3 selects NORTH (RM+RN). The course definition and the administrative layer demonstrated in the 26 October lecture determine these weather sampling polygons. They approximate regional weather exposure and should not be described as exact hydrological catchment boundaries. The original NVE catalogue is retained

separately: it lists four codes, whereas the observed statistics contain three. This metadata discrepancy does not prevent the app from using the three coursework weather regions.

Sources: NVE electricity areas, NVE administrative regions, and the Canvas assignment.

NVE's handbook, section 3.1, states that weekly statistics are published for three watercourse areas. In the 26 October lecture, session 1, the visible demonstration exports NVE_Region at 8:42, shows RS/RØ/RV/RM/RN at 9:00, and samples the shapefile with GeoPandas at 11:00-12:00. These are visual checks; the recording's audio was not transcribed. The app uses VASS1=EAST, VASS2=WEST and VASS3=NORTH as the coursework association. The visual lecture checks alone do not independently verify this code-to-region association.

Code cell 2

```
points = pd.read_csv(project_root / 'data/regions/sampled_locations.csv')
counts = points.groupby(['region_type', 'region']).size()
assert counts.eq(1000).all()
display(counts.rename('Locations').to_frame())
areas = gpd.read_file(project_root / 'data/regions/electricity.geojson').to_crs(3035)
no3 = areas.loc[areas.ElSpotOmr.eq('NO3')]
locations = points.loc[points.region_type.eq('electricity') & points.region.eq('NO3')]
sample = gpd.GeoDataFrame(locations, geometry=gpd.points_from_xy(locations.longitude, locations.latitude),
crs=4326).to_crs(3035)
assert sample.within(no3.geometry.iloc[0]).all()
ax = no3.plot(color='#eef3f7', edgecolor='#284b63', figsize=(7, 8))
sample.plot(ax=ax, markersize=3, color='#2274a5', alpha=0.6)
ax.set_title('NO3: 1,000 sampled locations')
ax.set_axis_off()
plt.show()
```

		Locations
region_type	region	
assignment_precipitation	EAST	1000
assignment_precipitation	NORTH	1000
assignment_precipitation	WEST	1000
electricity	NO1	1000
electricity	NO2	1000
electricity	NO3	1000
electricity	NO4	1000
electricity	NO5	1000
nve_regions	RM	1000
nve_regions	RN	1000
nve_regions	RS	1000
nve_regions	RV	1000
nve_regions	RØ	1000

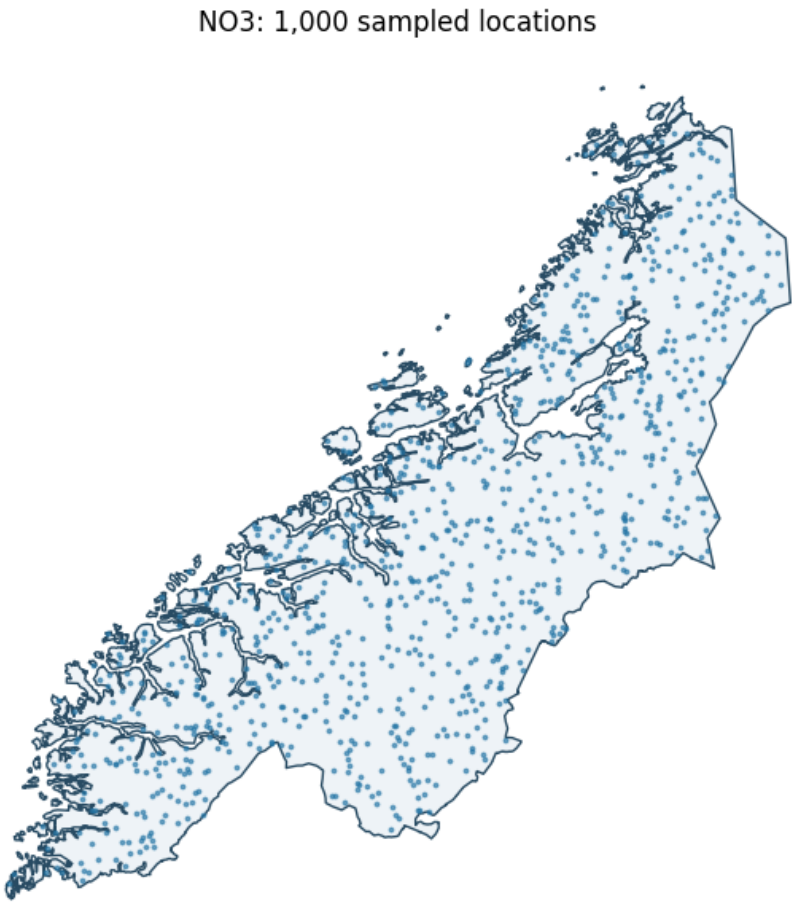


Figure 1. NO3: 1,000 sampled locations.

Code cell 3

```
from ind320.area_mapping import weather_region, PRECIPITATION_MEMBERS
unions = gpd.read_file(project_root / 'data/regions/assignment_precipitation.geojson').set_index('region')
for number, expected_region in [(1, 'EAST'), (2, 'WEST'), (3, 'NORTH')]:
    family, region = weather_region('VASS', number)
    selected = points.loc[points.region_type.eq(family) & points.region.eq(region)]
    assert region == expected_region and len(selected) == 1000
    geometry = gpd.GeoSeries(gpd.points_from_xy(selected.longitude, selected.latitude), crs=4326)
    assert geometry.within(unions.loc[region, 'geometry']).all()
    print(f'VASS{number} → {region}: {" + ".join(PRECIPITATION_MEMBERS[region])}; 1,000 points verified')
```

Output

```
VASS1 → EAST: RS + RØ; 1,000 points verified
VASS2 → WEST: RV; 1,000 points verified
VASS3 → NORTH: RM + RN; 1,000 points verified
```

2. Hourly weather from ERA5

`ind320.weather_data.download_weather(region, start_date, end_date)` provides a direct location-based download function. A two-day pilot at sampled NO3 point 0 returned 48 hourly temperature and precipitation values without missing observations. It used the API's default elevation handling and was only a connection test.

The complete regional workflow is in `ind320.era5_region.regional_weather(region, start_date, end_date)`. It explicitly selects ERA5, UTC, Celsius and millimetres. ERA5's grid spacing is 0.25°. For this analysis, nearest-cell selection is used and point-specific elevation downscaling is disabled with `elevation=nan`. This preserves a consistent grid-scale reanalysis interpretation; fine-scale terrain temperature differences are not represented.

The 1,000 NO3 points occupy 236 ERA5 cells. One representative original coordinate is requested for each occupied cell. Each returned cell centre must match the expected grid cell. Its series is weighted by the number of original sampled points in that cell. This is equivalent to averaging all 1,000 point assignments, rather than assigning equal weight to 236 cells. They are not 1,000 independent weather stations.

Responses are cached independently, and a local usage ledger limits weighted API calls below the provider's published limits. A 429 response stops the download. Complete hourly timestamps, units, non-negative precipitation and full spatial coverage are checked before a regional dataset is saved. The raw responses remain local; the regional means and source manifest are included in GitHub.

The file below covers 1 January to 31 December 2024, including leap day: 8,784 UTC hours. The notebook reads that verified result so rerunning the analysis does not repeat the download. To refresh it, run `python -m ind320.era5_region` from the project folder.

Data: Open-Meteo Historical Weather API, using Copernicus ERA5. Licence: CC BY 4.0. Regional means and subsequent analyses are derived products. Rate accounting was checked against the provider's pricing documentation and public query-weight calculation.

Code cell 4

```
weather_path = project_root / 'data/weather/electricity_NO3_2024-01-01_2024-12-31.csv'
weather = pd.read_csv(weather_path, parse_dates=['time_utc']).set_index('time_utc')
source = json.loads(weather_path.with_suffix('.source.json').read_text())
expected = pd.date_range('2024-01-01', '2025-01-01', freq='h', inclusive='left', tz='UTC')
assert weather.index.equals(expected)
assert weather.sample_count.eq(1000).all()
assert weather[['temperature_2m', 'precipitation']].notna().all().all()
assert sum(c['sample_weight'] for c in source['cells']) == 1000
assert len(source['cells']) == 236
print('Verified hours:', len(weather))
print('Sampled locations: 1,000; occupied ERA5 cells:', len(source['cells']))
display(weather[['temperature_2m', 'precipitation']].describe())
```

Output

```
Verified hours: 8784
Sampled locations: 1,000; occupied ERA5 cells: 236
```

	temperature_2m	precipitation
count	8784.000000	8784.000000
mean	4.678415	0.199975
std	7.561637	0.287608
min	-15.913000	0.000000
25%	-0.317750	0.009400
50%	4.283150	0.084800
75%	10.431550	0.278525
max	24.481900	3.354400

3. Precipitation: DCT and robust SPC limits

This section analyses precipitation. The control limits are calculated from the filtered precipitation residuals.

`precipitation_spc()` applies an orthonormal DCT-II. Frequencies below the cut-off form a smooth baseline; subtracting this gives the high-pass residuals. Their full-year median and normal-consistent median absolute deviation provide a robust centre and scale. The moving limits on the original precipitation plot are baseline + residual median ± k × robust scale. Only the original observations, limits and flagged values are plotted.

Defaults are a cut-off of 1/30 cycles per day and k=3. Both are parameters. The scale conversion does not make rainfall Gaussian: skewness, dry periods and serial dependence remain. The flags identify values unusual under

this rule, not confirmed errors or a nominal false-alarm probability. No flagged values are removed.

Method: SciPy DCT. The tests include a known injected spike and a constant dry series.

Code cell 5

```
spc = precipitation_spc(weather.precipitation, cutoff_cycles_per_day=1/30, deviations=3)
assert spc.data.index.equals(weather.index)
assert (spc.data.upper_mm >= spc.data.lower_mm).all()
display(pd.Series(spc.summary, name='SPC result').to_frame())
display(spc.data.loc[spc.data.outlier].head())
plt.show()
```

	SPC result
observations	8784
outliers	639
outlier_fraction	0.072746
robust_sigma_mm	0.159744
cutoff_cycles_per_day	0.033333
deviations	3
degenerate_scale	False

	precipitation_mm	lower_mm	upper_mm	outlier
time_utc				
2024-01-10 17:00:00+00:00	0.5481	-0.418269	0.540198	True
2024-01-12 21:00:00+00:00	0.6671	-0.389935	0.568532	True
2024-01-12 22:00:00+00:00	0.8132	-0.389336	0.569131	True
2024-01-12 23:00:00+00:00	0.9714	-0.388735	0.569732	True
2024-01-13 00:00:00+00:00	1.1789	-0.388132	0.570334	True

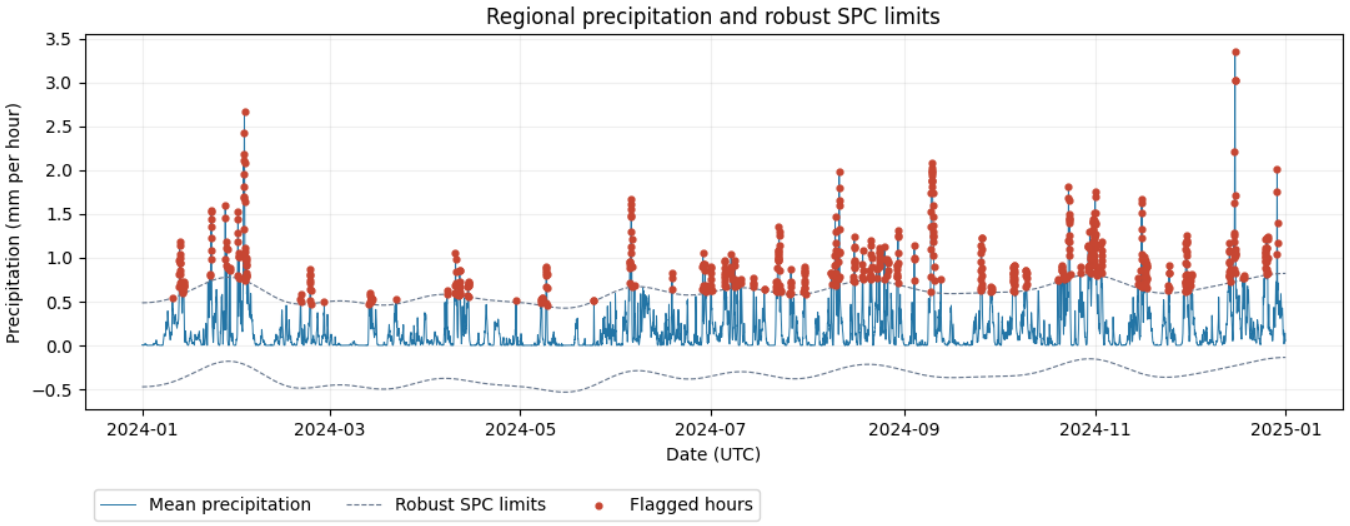


Figure 2. Regional precipitation, robust SPC limits and flagged hours.

With these defaults, the rule marks 639 hours, about 7.27% of the year. This is much higher than a Gaussian three-sigma tail rate, which reinforces that such a probability interpretation is unsuitable for this precipitation series. Changing the cut-off or multiplier changes the result; the app exposes both controls.

4. Temperature: Local Outlier Factor

`temperature_lof()` uses temperature and its change from the preceding hour as two features. Standardisation prevents their different scales from dominating distance. LOF compares local densities among feature-space neighbours; it does not automatically use neighbours adjacent in time.

The default contamination is 1%, with 20 neighbours. The initial hour has no preceding change and is explicitly excluded from scoring. A target proportion sets a score threshold; it does not establish that this percentage of observations is erroneous. The model is fitted and evaluated on this historical dataset, rather than used to predict future anomalies.

Method: scikit-learn LocalOutlierFactor. A synthetic isolated extreme is checked in the numerical tests.

Code cell 6

```
lof = temperature_lof(weather.temperature_2m, contamination=0.01, neighbours=20)
assert not lof.data.evaluated.iloc[0]
assert lof.summary['evaluated'] == 8783
display(pd.Series(lof.summary, name='LOF result').to_frame())
display(lof.data.loc[lof.data.anomaly].sort_values('lof_score', ascending=False).head())
plt.show()
```

	LOF result
observations	8784.000000
evaluated	8783.000000
anomalies	88.000000
requested_fraction	0.010000
actual_fraction	0.010019
neighbours	20.000000
excluded_initial_rows	1.000000

	temperature_c	change_c	lof_score	evaluated	anomaly
time_utc					
2024-02-09 16:00:00+00:00	-11.2879	-1.1680	2.102789	True	True
2024-02-09 15:00:00+00:00	-10.1199	-1.1467	2.054219	True	True
2024-02-07 16:00:00+00:00	-12.3263	-1.0839	1.851994	True	True
2024-04-21 19:00:00+00:00	1.1768	-2.0179	1.851651	True	True
2024-05-22 06:00:00+00:00	14.0179	2.3803	1.790812	True	True

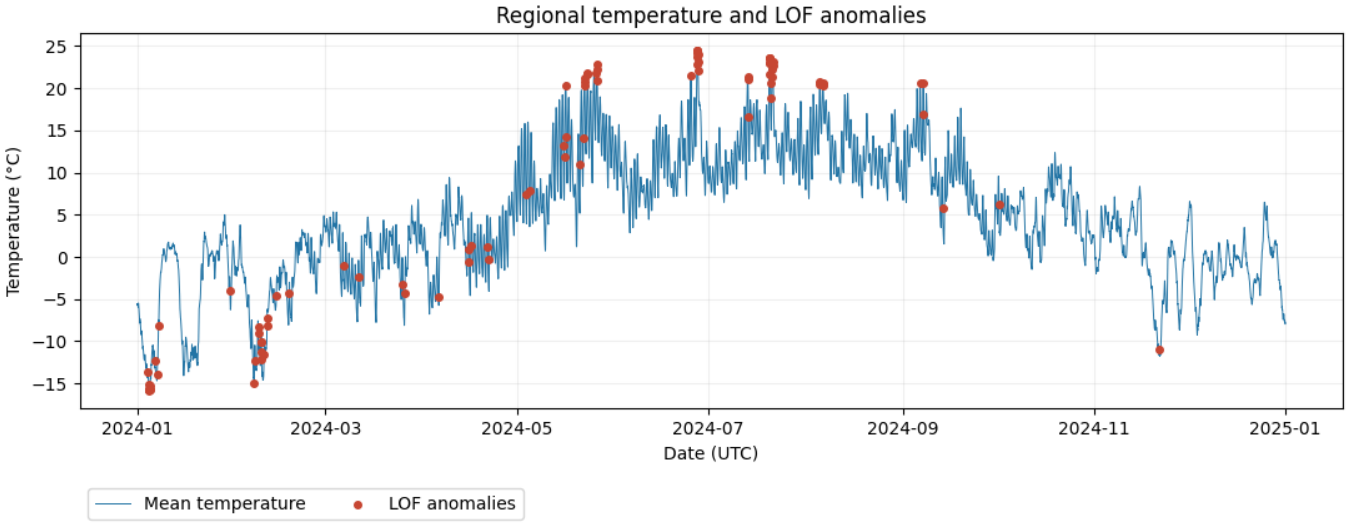


Figure 3. Regional temperature and Local Outlier Factor anomalies.

The default threshold marks 88 of the 8,783 evaluated hours, approximately 1%. This count is largely determined by the selected contamination. Its interpretation comes from the feature definition and the observations flagged, rather than from the count alone.

5. Reservoir filling: STL decomposition

The NVE snapshot contains weekly filling ratios. Both the observation date and area type/number are used to select EL 3. The series has 1,659 observations from 8 January 1995 to 18 October 2026, with no missing weeks or filling values. No interpolation is needed.

`filling_stl()` returns the plot, component table and settings. Defaults are a 52-week period, seasonal smoother 13, trend smoother 81 and robust fitting enabled. The period approximates a calendar year; it does not explicitly model occasional ISO week 53. Seasonal and trend smoothers must be odd, and the trend smoother must exceed the period. The app exposes these parameters.

The components remain in filling fractions, not percentage points. The residual captures variation unexplained by this decomposition and should not automatically be labelled a data error.

Source: NVE Magasinstatistikk API. Method: statsmodels STL.

Code cell 7

```
reservoirs = pd.read_csv(project_root / 'data/reservoirs/nve_snapshot.csv', parse_dates=['date'])
filling = reservoirs.loc[reservoirs.area_type.eq('EL') &
reservoirs.area_number.eq(3)].set_index('date').filling_ratio
assert filling.index.to_series().diff().dropna().eq(pd.Timedelta(days=7)).all()
assert filling.notna().all()
stl = filling_stl(filling, seasonal=13, trend=81, robust=True)
assert stl.summary['max_reconstruction_error'] < 1e-10
display(pd.Series(stl.summary, name='STL result').to_frame())
plt.show()
```

	STL result
observations	1659
period_weeks	52
seasonal	13
trend	81
robust	True
max_reconstruction_error	0.0

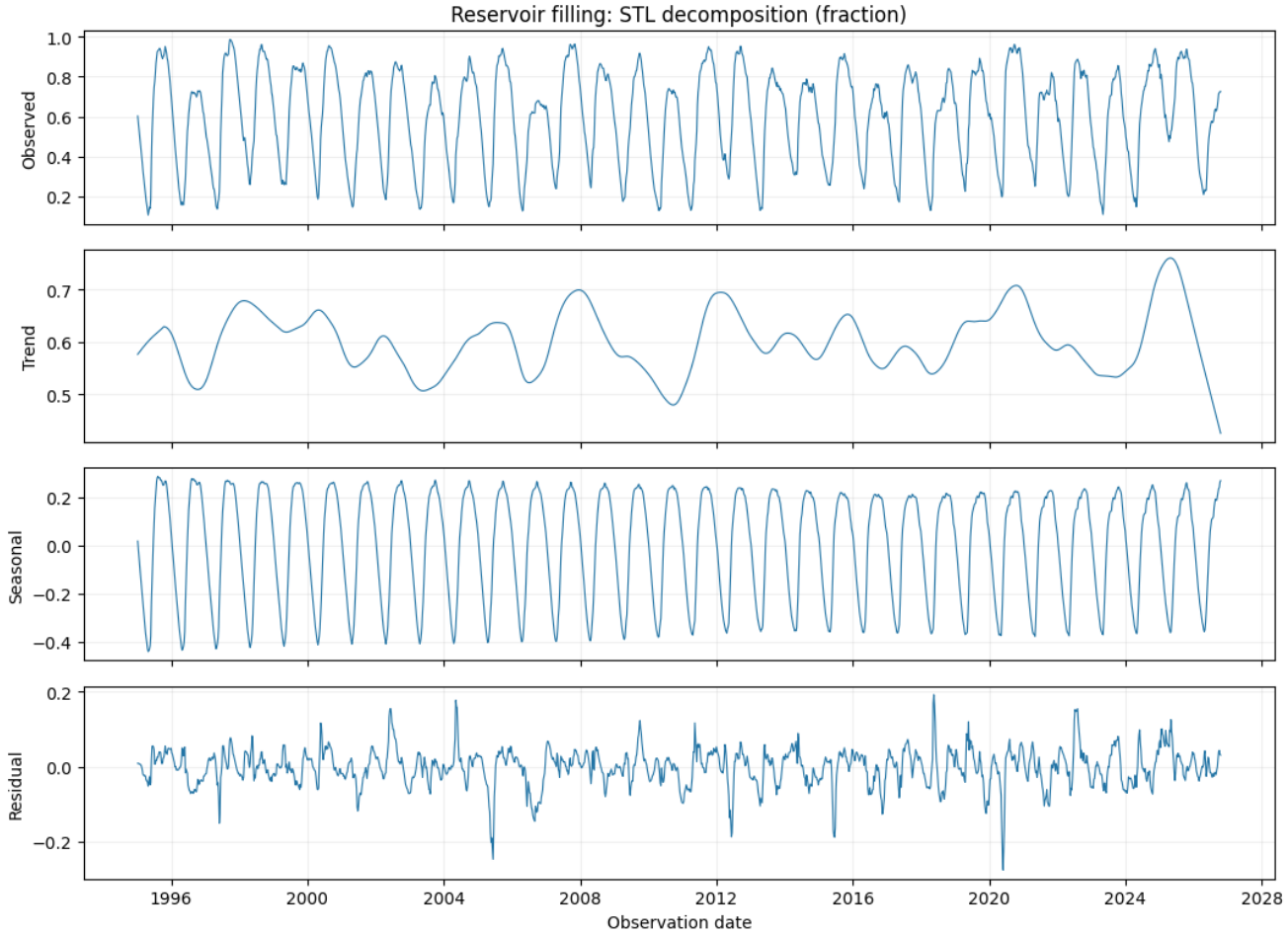


Figure 4. STL decomposition of the weekly EL 3 filling series.

The observed series is reconstructed from trend + season + residual to floating-point precision. This verifies the component accounting, not that the chosen decomposition is uniquely correct. The plot helps distinguish recurrent annual variation from changes over several years and shorter departures.

6. Reservoir filling: spectrogram

`filling_spectrogram()` uses the same EL 3 filling series as STL. A Hann window of 104 weeks and overlap of 78 weeks are the defaults. Linear detrending is applied within each window. The axes show window-centre dates and cycles per 365.25-day year. Power density is displayed in decibels relative to fraction² per cycle/year.

A longer window improves frequency resolution at the cost of time localisation. Here, the resolution is about 0.502 cycles/year and there are 60 windows. The 78-week overlap makes adjacent windows strongly dependent. Colour differences show changing spectral power, not separate anomaly classifications.

The teaching implementation uses the established SciPy spectrogram function. SciPy now classifies it as legacy and offers ShortTimeFFT for newer workflows. A synthetic annual signal is used to check frequency calibration.

Code cell 8

```
spectrum = filling_spectrogram(filling, window_length=104, overlap=78)
assert spectrum.data.shape[0] == spectrum.summary['windows']
assert np.isfinite(spectrum.data.to_numpy()).all()
display(pd.Series(spectrum.summary, name='Spectrogram result').to_frame())
plt.show()
```

	Spectrogram result
window_weeks	104.000000
overlap_weeks	78.000000
windows	60.000000
frequency_resolution_per_year	0.501717

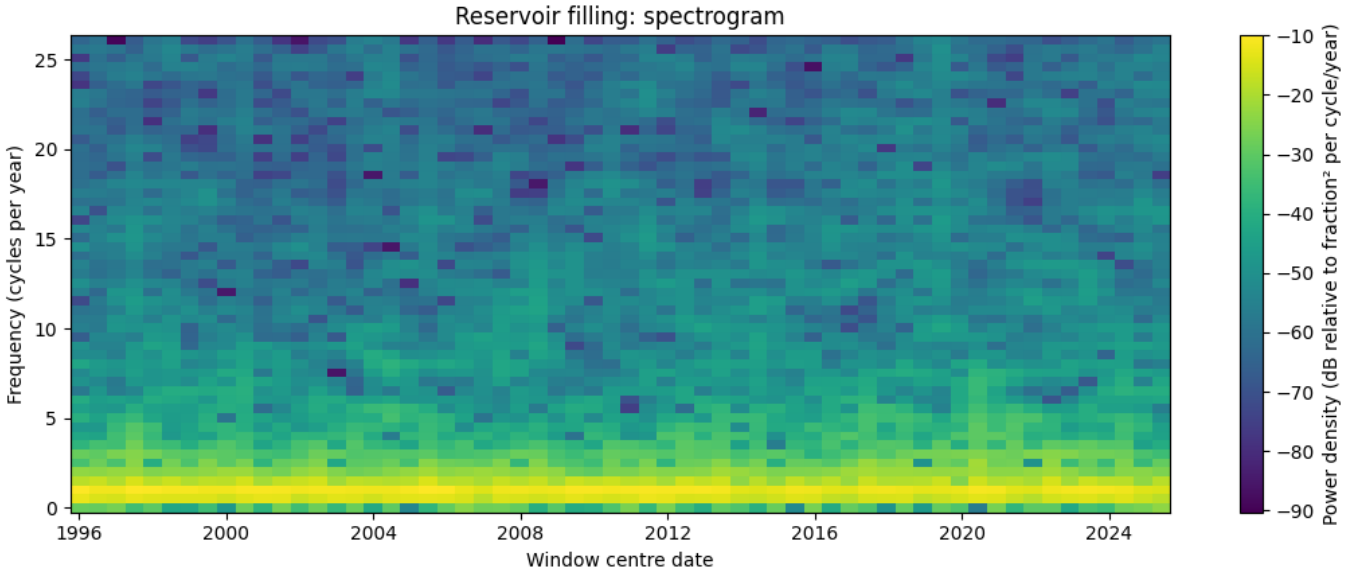


Figure 5. Spectrogram of the same weekly filling series.

7. Function tests and Streamlit integration

The tests use known examples: a rainfall spike, a dry series, an isolated temperature extreme and a repeating annual pattern. They also check that STL components reconstruct the original series, that missing values are rejected, and that weather cells keep the correct sample weights. Cached weather responses are reused instead of downloaded again.

The app uses the same analysis functions as the notebook. **Reservoir patterns** has tabs for STL and the spectrogram. **Weather data quality** has tabs for SPC and LOF. Each tab has controls for its settings. The area type is shared between pages, while each analysis page has its own area selector.

The complete NO3 weather dataset for 2024 is saved with the project. Other regions can be downloaded from Open-Meteo, but their full datasets have not all been checked. The existing NVE reservoir pages and MongoDB electricity-transfer page are retained. Area comparison is now part of Plots.

All 12 offline tests passed. Streamlit checks covered both new pages, parameter changes and the region selectors. The published app was also checked separately. Part 3 was developed on a separate branch and merged into main, using the same public app address as before.

Code cell 9

```
import unittest
from tests import test_quality_analysis, test_era5_region
suite = unittest.TestSuite([
    unittest.defaultTestLoader.loadTestsFromModule(test_quality_analysis),
    unittest.defaultTestLoader.loadTestsFromModule(test_era5_region),
])
checks = unittest.TextTestRunner(verbosity=1).run(suite)
assert checks.wasSuccessful()
print('Numerical and weighting tests passed:', checks.testsRun)
```

Output

```
......
----------------------------------------------------------------------
Ran 6 tests in 0.370s

OK
Numerical and weighting tests passed: 6
```

8. AI usage

This project combines my own work and wording with substantial AI assistance from OpenAI Codex (Astra 6). The tool helped organise the repository, retrieve data, develop and test the Python code, and prepare the Streamlit pages. The explanations and work log combine my own formulations with AI-assisted drafting and language revision. AI was therefore used for both technical development and writing, rather than proofreading alone. Codex also helped edit my own screencast recordings into a single video; the narration is my own.

9. Work log

Part 3 builds on the reservoir and electricity-transfer app from the earlier projects. The main additions are hourly weather data and four ways to explore unusual values and recurring patterns. The notebook records the methods and results, while the app lets the reader change the settings and compare the plots.

The first task was to prepare the regions for weather sampling. The normal NVE download portal was unavailable, so the published map services were used to retrieve the boundaries and export shapefiles. Each region received 1,000 random locations. The east and north administrative regions were combined before sampling, so smaller component regions would not receive too much weight.

A short weather request was used to check the API connection, timestamps and units. The full NO3 download then covered all 8,784 hours of 2024. Its 1,000 sampled locations shared 236 ERA5 grid cells. Each cell was downloaded once, but kept the weight of all the points assigned to it. This was the main challenge in preparing the regional average. Simply averaging the 236 cells would have changed what the original sample represented.

For precipitation, DCT filtering was used to find short-term changes, followed by robust SPC limits. For temperature, Local Outlier Factor compared temperature and its change from the previous hour. STL and the spectrogram used the same weekly reservoir filling series. Tests with known patterns and isolated extremes helped check that the functions behaved as expected.

One useful lesson was that a flagged value is not automatically wrong. LOF marks roughly the selected proportion of observations, while SPC changes when its cut-off or limits change. The plots therefore keep all observations and explain what the flags mean.

In Streamlit, the analyses were grouped into two pages with tabs and controls. The existing reservoir and electricity-transfer pages were kept. Shared Python functions were moved into one package, and reports, recordings and tests were placed in separate folders. The app was checked locally and online after the branch was merged. The screencast shows the app and the notebook results. As described in the AI section, the project combines my own contributions with Codex assistance in code, testing and writing.

10. Screencast

Download the Part 3 screencast (MP4). Open the downloaded file to watch it.

Duration: 3 minutes 10 seconds. The video demonstrates regional sampling, SPC and LOF controls, STL and the spectrogram, and the notebook test results.